

Predicting Reasoning-Model Costs from Shared Prefill Activations

Anonymous authors

Abstract

*Routers typically choose a model per query by trading predicted correctness against cost. However, most price each model at a fixed cost, assuming that the number of tokens each model outputs does not vary significantly. We find that the output of reasoning models varies by an order of magnitude across queries. We predict **both** success probability **and** route-specific output length from the prefill activations of a single small frozen 4B LLM, using a linear readout per route. On 341 LiveCodeBench, 1,000 Omni-MATH and 6,500 MMLU-Pro tasks, predicted costs reduce spending at matched accuracy by 33.7%, 34.8% and 28.8% relative to median-length pricing. They also beat every alternative estimator we evaluated, with paired intervals excluding zero. We then ablate across different small LLMs and show that most small LLM prefills can be used as routers in this way.*

1. Introduction

Routing reasoning models requires predicting both correctness and cost before generation. Token rates are published; the number of tokens is not. A model that is cheap on average can be expensive on a particular problem, and the outputs of the models we study vary 10–90 $\times$ across problems.

Prefill routers predict correctness from a small encoder’s prompt activations; the prefill router of Varshney et al. [10] explicitly prices outputs at median training length. We read output length from the same activations (Fig. 1). Cost prediction adds one linear readout per route to an existing prefill router, without another encoder pass.

We contribute: (i) evidence on fresh problems and billed prices that prefill cost readouts reduce spending at matched accuracy and outperform mean-length, prompt-feature, text-embedding and ZeroRouter cost estimators; (ii) a mechanism, from ablations that price from our own success readouts: difficulty suffices where output length tracks it and fails where it does not; (iii) controls showing the gain is the prefill representation, holds from a 1.7B prefill upward and transfers to two further model families; (iv) a deployment finding: the same model’s endpoints drift, and

the shared readouts absorb this with one offset per endpoint.

Related work. MixLLM predicts output length from query embeddings [11]; CARROT estimates cost with embedding nearest neighbours or a fine-tuned RoBERTa [9]; ZeroRouter prices queries by item-response difficulty bins [14]. Activation-based length prediction also supports scheduling [8, 13]. Hosted open-weight models vary across providers in accuracy, length and price [2, 6, 7]; FACET routes among providers of a fixed model per (provider, task) using measured outcomes [3]. We instead route per query over endpoints of several models with predicted, endpoint-specific costs.

2. Cost Prediction and Routing

For query x and route m (a model at a fixed reasoning effort, served by an endpoint), let $p_m(x)$ be the probability of a correct answer and $c_m(x)$ the expected generation cost. One route is chosen before generation,

$$m^*(x; V) = \arg \max_m \{V \hat{p}_m(x) - \hat{c}_m(x)\}, \quad (1)$$

where V is the value of a correct answer; sweeping V trades accuracy for cost. Every query is answered once, with no verifier or fallback.

Readouts. We concatenate mean and last-token activations from eight layers of a frozen Qwen3-4B (Instruct-2507; Thinking-2507 on Omni), standardized on training data. Success readouts are L2-regularized logistic regressions with held-out Platt calibration; cost readouts are ridge regressions of log mean output length, regularized by training-only cross-validation. Predicted length is exponentiated with residual smearing and matched to the route’s training mean, then priced: $\hat{c}_m(x) = r_m^{\text{in}} n_m^{\text{in}}(x) + r_m^{\text{out}} \hat{\ell}_m(x)$.

Endpoint offsets. An endpoint of an already-modelled route (a new provider, or a provider whose behaviour changed) reuses the route’s readouts with one multiplicative length offset, the ratio of observed to predicted mean output over k labelled calls, and its own price.

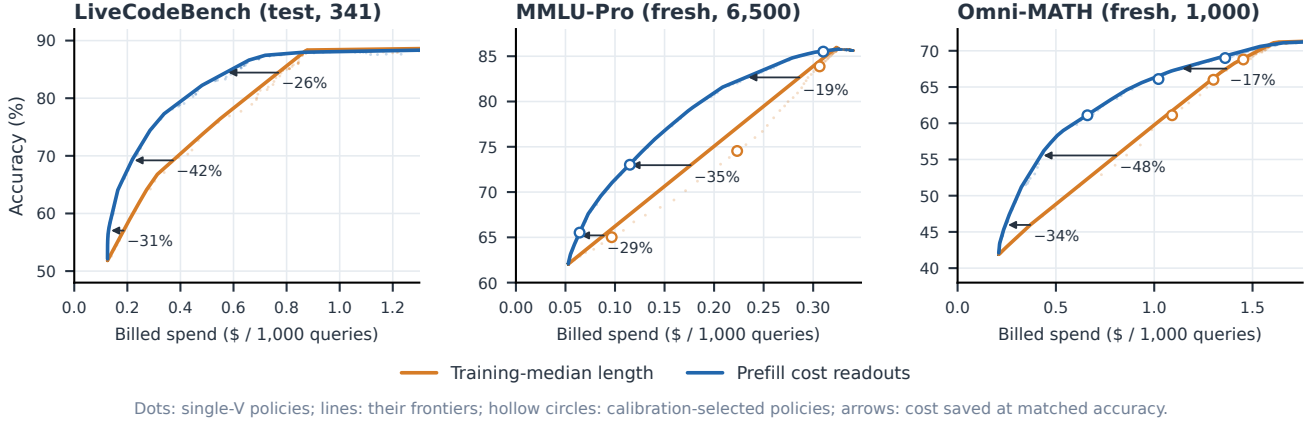


Figure 1. **Cost-accuracy frontiers at billed prices.** Prefill cost readouts (blue) versus training-median length (orange), with the same success predictions and the same encoder pass. Dots: the deterministic policy at each V ; lines: their frontiers; hollow circles: policies selected on original calibration for the targets of Table 2. Arrows run from the median rule’s frontier to ours at matched accuracy and give the cost saved; averaged over the shared accuracy band the saving is 34% on LiveCodeBench, 29% on MMLU-Pro and 35% on Omni-MATH.

3. Experimental Protocol

The routes are gpt-oss-20b at low and medium effort, deepseek-v4-flash, and gpt-oss-120b at medium and high effort, called through OpenRouter, with deepseek-v4-flash pinned to one provider (StreamLake) throughout, because its output length depends on the serving provider (see End-points below). Original pools: 892 LiveCodeBench [5], 500 Omni-MATH-500 [1] and 1,000 subject-stratified MMLU-Pro [12] problems with 2–16 draws per route; train/calibration/test splits are 441/110/341, 275/75/150 and 550/150/300. Fresh sets: 6,500 MMLU-Pro and 1,000 Omni-MATH problems disjoint from the original pools (one draw per route), collected after all readouts were fitted.

Prices. Realized cost is the cost billed for each call (on LCB, realized tokens at the billed rates). Predictions are priced at each model’s effective billed rate, fitted on the fresh calls (USD per million output tokens: 0.13 for gpt-oss-20b, 0.08 for pinned deepseek-v4-flash, 0.26 for gpt-oss-120b). These differ from the list prices by up to $2.3\times$. Rates cross-fitted on the other half of the fresh problems move by at most 1.5% and change every comparison below by at most 0.6 points, so fitting rates on the evaluation calls does not favour any estimator.

Evaluation. All estimators are fitted on original training problems only and evaluated on LCB’s temporal test split and on the fresh problems, with the same success predictions. Each V on a dense grid gives a deterministic policy and an observed (cost, accuracy) point. Comparing

Cost saved by ours vs.	LCB ($n=341$)	Omni-MATH ($n=1,000$)	MMLU-Pro ($n=6,500$)
Training-median length	33.7 [24.1, 39.9]	34.8 [27.9, 39.8]	28.8 [26.0, 31.8]
Training-mean length	29.6 [23.6, 34.5]	35.8 [28.9, 40.5]	28.6 [25.9, 31.7]
Prompt-feature GBM	26.3 [16.3, 32.9]	26.5 [18.9, 31.9]	25.2 [21.0, 28.7]
MixLLM-style embeddings	18.2 [11.2, 25.2]	26.7 [20.0, 32.5]	19.2 [15.3, 22.5]
ZeroRouter (reimplemented)	21.1 [13.2, 26.8]	14.4 [8.4, 20.6]	29.0 [25.5, 32.0]
<i>Ablations: price from our success readouts</i>			
Difficulty bins	4.0 [−1.1, 9.0]	6.4 [1.1, 11.6]	22.3 [18.8, 24.7]
Cost from success	−0.8 [−5.6, 4.0]	−0.1 [−4.6, 4.6]	24.1 [21.1, 26.7]

Table 1. **Cost saved at matched accuracy, billed costs.** Cost saved (%) by our prefill cost readouts relative to each estimator, with 95% paired bootstrap intervals; same success predictions for every arm except ZeroRouter, which uses its own. LCB: the 341-problem temporal test split; Omni-MATH and MMLU-Pro: fresh problems. MixLLM-style: jina-embeddings 137M with the MixLLM ensemble head (LCB, code-aware variant). ZeroRouter: item-response latent fitted on the five routes, 4B features, its own success model and bin pricing, configuration chosen on calibration. The two ablations price each route from our own success readouts: ten quantile bins of mean success logit (ZeroRouter’s pricing rule), or a ridge regression of log length on the success logits.

two cost estimators a, b , we interpolate each one’s cost at 12 accuracies spanning the interior 90% of the range both reach and report $1 - \exp(\log C_a/C_b)$, the cost saved by a at matched accuracy. Intervals are percentile intervals from paired problem bootstraps (300 resamples), keeping each problem’s routes together. Separately, deployable policies select the cheapest V reaching each accuracy target on original calibration and are applied once to the fresh problems.

Fresh set	Target	Savings, %	Δ acc., pp	Matched acc., %
MMLU-Pro	0.65	33.5 [27.6, 38.5]	+0.51 [−0.12, +1.18]	30.2 [25.8, 34.2]
	0.75	48.5 [44.2, 52.3]	−1.52 [−2.62, −0.52]	35.0 [30.2, 39.2]
	0.85	−1.2 [−2.9, 0.5]	+1.63 [+1.18, +2.08]	2.5 [1.1, 3.9]
Omni-MATH	0.65	39.5 [33.4, 45.2]	+0.00 [−2.30, +2.30]	37.8 [29.0, 43.0]
	0.70	21.5 [16.0, 27.0]	+0.10 [−1.70, +2.00]	21.1 [12.9, 27.4]
	0.75	6.3 [2.9, 9.9]	+0.20 [−1.10, +1.40]	6.7 [−0.5, 11.1]

Table 2. **Deployable policies.** Policies chosen on original calibration for each accuracy target and applied once to fresh problems; learned versus median-length pricing, billed costs. Savings and Δ accuracy compare the two selected policies (paired bootstrap, 2,000); matched accuracy compares our policy with the median-pricing frontier at our achieved fresh accuracy (300 resamples). Fresh accuracy need not equal the target.

4. Results

Cost readouts beat every estimator. At matched accuracy, prefill cost readouts spend 33.7% less than median-length pricing on LCB, 34.8% less on Omni-MATH and 28.8% less on MMLU-Pro (Table 1). Every external estimator loses with intervals excluding zero: mean length by 29–36 points, a prompt-feature model by 25–27, MixLLM-style embeddings by 18–27 and a full ZeroRouter reimplementation by 14–29. Calibration-selected policies save 21–49% in the middle of the range at accuracies within 1.6 points of the median policy; at matched accuracy they save 21–38% there and 2–7% near the top, where every method calls the strongest routes (Table 2). Calibration selection costs little: our selected policies spend 0–4% more than our own fresh curve at the same accuracy.

When difficulty suffices. Two ablations price each route from our own success readouts instead of a dedicated cost readout: ZeroRouter’s bin lookup on mean success logit, and a ridge regression of log length on the success logits. Both tie the cost readout on LCB and Omni-MATH, where output length tracks difficulty, and lose by 22–24 points on MMLU-Pro. There, even empirical difficulty explains little of log length ($R^2 \approx .19$), while subject labels recover roughly 60% of the gap. The prefill encodes length information beyond difficulty, and that information pays where difficulty and length come apart.

Representation, readout and prefill size. Table 3 crosses three text encoders with three cost readouts: ridge, CARROT-style k -nearest neighbours and a MixLLM-style ensemble, each with our success predictions and with every encoder pass priced. Our readout saves 16–27% against every cell, including Qwen3-Embedding-8B, an encoder twice the prefill’s size. On PCA-reduced 4B features the three readouts are within about 7 points of one another, so the gain comes from the representation rather than the regressor. Pricing the encoder pass changes no comparison

Encoder	Omni-MATH			MMLU-Pro		
	Ridge	k NN	MixLLM	Ridge	k NN	MixLLM
Qwen3-Embedding-8B	16.6	20.9	21.5	21.1	21.3	19.5
jina-embeddings 137M	23.8	25.3	26.6	17.8	20.4	18.9
MiniLM-L12	24.3	25.7	25.4	18.8	19.0	16.2

Table 3. **Representation and readout.** Cost saved (%) by our prefill readouts relative to each encoder–readout cell at matched accuracy; fresh problems, billed costs, every encoder pass priced, same success predictions. Every paired bootstrap interval (200 resamples) excludes zero.

by more than 3 points. Table 4 varies the prefill model while keeping prompts, layers and both readouts fixed. Every size saves 18–28% against median pricing; length R^2 rises with size, but routing gains saturate by 1.7B, which is within the intervals of our 4B router. The 0.6B prefill loses 5–8 points and 8B adds nothing. The result is not specific to Qwen: prefills from three other families (Phi-4-mini, Granite-3.3-2B, SmoLLM2-1.7B) also save 20–28% against median pricing on both sets. Matching our router depends on the pool: Phi-4-mini is 3.2 points ahead on MMLU-Pro but 9 behind on Omni-MATH, and SmoLLM2-1.7B is within the intervals on Omni-MATH but 3.8 behind on MMLU-Pro.

New model families. Readouts for Qwen3-32B and GLM-4.7-flash, fitted on the same frozen 4B features, were evaluated on 1,905 fresh MMLU-Pro problems (fresh log-length R^2 .48 and .06). With all seven routes, prefill cost readouts save 30.0% [25.4, 35.3] relative to median pricing, 21.8% [16.3, 26.7] relative to cost from success and 20.6% [15.3, 25.4] relative to difficulty bins. On a pool of the two gpt-oss-20b routes and the two new families, they save 32.2%, 11.6% and 14.8%. Readouts onboarded from ten examples per new route match the full readouts in the seven-route pool (0.1 points) and trail them by 11.9 [−4.2, 21.9] in the new-family pool.

Endpoints and provider variability. On the fresh collection, the readouts overestimated deepseek-v4-flash’s cost by 37% (MMLU-Pro) and 23% (Omni-MATH); all other routes were within 8%. The aggregator had moved most of these calls to a provider absent from the original data, which writes about half as many tokens at equal accuracy. One length offset from 50 fresh calls removes the bias on average (predicted/realized 0.97 and 1.00 over 200 draws of the calls), but a single fit is noisy where lengths are heavy-tailed: 95% of fits land in [0.53, 1.51] on MMLU-Pro and [0.78, 1.25] on Omni-MATH. Pinning deepseek-v4-flash to each of three providers on 1,995 fresh MMLU-Pro and 957 APPS problems shows that an endpoint is close to its model plus an offset: providers agree on correctness

Prefill	Omni-MATH			MMLU-Pro		
	R^2	vs. median	vs. ours	R^2	vs. median	vs. ours
Qwen3-0.6B	.60	24.6	−5.1	.23	18.0	−7.5
Qwen3-1.7B	.64	27.4	−3.4	.33	23.5	+0.5
Qwen3-4B (hybrid)	.66	24.7	−1.7	.36	22.2	−3.3
Qwen3-8B	.67	24.1	+0.3	.37	20.8	−2.2
Phi-4-mini (3.8B)	.64	23.7	−9.2	.36	25.3	+3.2
Granite-3.3 (2.5B)	.61	21.3	−9.2	.29	22.1	−2.1
SmolLM2 (1.7B)	.58	28.4	−2.9	.23	20.5	−3.8
Qwen3-4B-2507 (ours)	.69	27.8	−	.39	23.1	−

Table 4. **Prefill size.** Each prefill feeds both readouts, refitted with the paper’s recipes on original training problems; fresh problems, billed costs. R^2 : fresh log-length R^2 averaged over routes. vs. median: cost saved relative to median pricing with the same row’s success predictions. vs. ours: cost saved by the row’s router relative to ours (negative: it spends more); its paired intervals (300 resamples) include zero for Qwen3-1.7B and 8B on both sets, SmolLM2 on Omni-MATH and Granite on MMLU-Pro.

for 90–96% of problems (73–78% if independent) and their output lengths correlate at .83–.95. The offsets mainly decide which provider to pin. Pinning the provider that is cheapest on the fitting problems spends 4.6% [0.4, 10.3] less than routing over all three on APPS and 3.9% [1.5, 6.7] less on MMLU-Pro; 50 calls per provider pick that provider 79% (MMLU-Pro) and over 99% (APPS) of the time, at an expected 2.1% and 0.05% extra cost. Provider mixing also hides predictable cost, which is why every result above pins deepseek-v4-flash. Recollecting all of its calls on one provider and refitting the readouts raises our saving over median pricing on every large set (same script on both sides): LCB from 26.7% to 34.5%, fresh Omni-MATH from 27.8% to 34.8%, fresh MMLU-Pro from 23.1% to 28.8%, AIME from 11.3% to 16.1% and APPS from 12.1% to 16.8%. On APPS the pinned route’s length R^2 rises from .08 to .64.

Live run. On 1,000 MMLU-Pro problems never used before, the frozen router called only its chosen routes. Token predictions held (predicted/realized 0.94–1.04 per route). With deepseek-v4-flash pinned, the calibration-chosen policies spent 25%, 19% and 4% less than median pricing at targets .65/.75/.85, while landing 1.1–2.1 points less accurate, so these are not matched-accuracy savings. Left unpinned, the aggregator sent deepseek-v4-flash calls to providers billing up to 14× the rate seen in the fresh collection.

Original pools and contrasts. On the original test sets, savings against median pricing are 17.5% [0.8, 32.7] on Omni-MATH (150 problems) and 43.9% [32.6, 52.3] on MMLU-Pro (300). On APPS they are 16.8% [5.6, 28.3]; on BigCodeBench, where output length is barely driven by

difficulty, 6.7% [−1.4, 13.0] against 13% oracle headroom. On AIME 1983–2024 (933 problems), our pre-registered screen predicted no gain from a single draw of the cheapest route (probe R^2 .13). Predicted costs instead save 14.9% [9.4, 21.4] at list prices (16.1% billed): with averaged draws the probe reads length at R^2 .39–.52, so the one-draw screen understated predictability and the call failed. Router-Bench [4] has 10.5% headroom.

5. Discussion and Limitations

Each route needs output-length labels, and an endpoint offset needs a few labelled calls; a single 50-call offset is noisy where lengths are heavy-tailed. The original pools use five routes from two model families; two further families (Qwen3-32B, GLM-4.7-flash) were tested on MMLU-Pro only. Fresh sets have one draw per route, and pinned providers were compared on APPS and MMLU-Pro only. Intervals are pointwise and assume independent problems, and Table 1 conditions on the fitted readouts; resampling the training problems as well widens them, most on the smaller Omni-MATH pool. Only deepseek-v4-flash is pinned to a provider; the gpt-oss routes are not, since their provider changes price but barely length. The prefill comparison covers four model families at up to 8B. Billed rates were measured over one collection, and prices change within days. The ZeroRouter, MixLLM and CARROT baselines are reimplementations. The original-pool pipeline was corrected after outcomes were observed.

6. Conclusion

A frozen prefill that already predicts success also predicts cost. On LiveCodeBench and on fresh problems at billed prices, its cost readouts reduce spending at matched accuracy and outperform every alternative estimator we tested; pricing from difficulty alone matches them only where output length tracks difficulty. Because endpoints of one model share these readouts up to an offset, a few dozen calls remove the bias that provider drift introduces, on average, and price a provider before it is pinned.

References

- [1] Bofei Gao, Feifan Song, Zhe Yang, et al. Omni-MATH: A Universal Olympiad Level Mathematic Benchmark For Large Language Models, 2024. arXiv:2410.07985.
- [2] Irena Gao, Percy Liang, and Carlos Guestrin. Model equality testing: Which model is this API serving? In *International Conference on Learning Representations*, 2025.
- [3] Liang He, Jingbo Wen, Yixiong Chen, Yue Yang, Qizhen Lan, Kangning Cui, and Xilu Wang. You cannot pick a provider from the price list: Market-aware routing for open-weight LLM inference, 2026.

- [4] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, et al. Router-Bench: A Benchmark for Multi-LLM Routing System, 2024. arXiv:2403.12031.
- [5] Naman Jain, King Han, Alex Gu, et al. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code, 2024. arXiv:2403.07974.
- [6] Haorui Li et al. When is the same model not the same service? a measurement study of hosted open-weight LLM APIs, 2026.
- [7] Guanjie Lin, Yinxin Wan, Shichao Pei, Ting Xu, Kuai Xu, and Guoliang Xue. Behavioral consistency and transparency analysis on large language model API gateways, 2026.
- [8] Glen Messenger. ALPS: Activation-Based Length Prediction for Intelligent LLM Inference Scheduling, 2026. Zenodo. doi:10.5281/zenodo.19078431.
- [9] Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, et al. CARROT: A Cost Aware Rate Optimal Router, 2025. arXiv:2502.03261.
- [10] Tanay Varshney, Annie Surla, Michelle Xu, et al. LLM Router: Rethinking Routing with Prefill Activations, 2026. arXiv:2603.20895.
- [11] Xinyuan Wang, Yanchi Liu, Wei Cheng, et al. MixLLM: Dynamic Routing in Mixed Large Language Models, 2025. arXiv:2502.18482.
- [12] Yubo Wang, Xueguang Ma, Ge Zhang, et al. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark, 2024. arXiv:2406.01574.
- [13] Huanyi Xie, Yubin Chen, Liangyu Wang, et al. Predicting LLM Output Length via Entropy-Guided Representations, 2026. arXiv:2602.11812.
- [14] Cheng Yan, Wuyang Zhang, Zhiyuan Ning, et al. Breaking Model Lock-in: Cost-Efficient Zero-Shot LLM Routing via a Universal Latent Space, 2026. arXiv:2601.06220.